Module 12: HAVING Clause

> The `HAVING` clause is used to filter groups created by the `GROUP BY` clause. While `WHERE` filters rows before grouping, `HAVING` filters groups after aggregation.

📖 Introduction

Consider an Employees table:

EmployeeID	Department	Salary
101	IT	50000
102	IT	60000
103	HR	45000
104	HR	55000
105	HR	65000

Management asks:

> Show only departments with more than 2 employees.

This cannot be done using a simple WHERE clause because the condition depends on an aggregate value (`COUNT(*)`).

For such situations, SQL provides:

```
HAVING
```

🎯 Learning Objectives

After completing this module, you will be able to:

✅ Understand HAVING

✅ Differentiate HAVING and WHERE

✅ Filter grouped data

✅ Use HAVING with COUNT()

✅ Use HAVING with SUM()

✅ Use HAVING with AVG()

✅ Use multiple HAVING conditions

✅ Create analytical reports

📚 Table of Contents

1️⃣ What is HAVING?

The `HAVING` clause filters groups after the `GROUP BY` operation has been performed.

Example

```
SELECT Department,
       COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 2;
```

Output:

Department	EmployeeCount
HR	3

Explanation:

```
IT → 2 employees
HR → 3 employees
```

Only HR satisfies:

```
COUNT(*) > 2
```

Why HAVING Matters

Used in:

- KPI Dashboards
- Sales Reports
- Revenue Analysis
- Customer Segmentation
- Business Intelligence

2️⃣ Why HAVING is Needed

Suppose you write:

```sql
SELECT Department,
       COUNT(*)
FROM Employees
WHERE COUNT(*) > 2
GROUP BY Department;
```

This causes an error.

Reason:

```
WHERE cannot use aggregate functions.
```

Correct:

```
SELECT Department,
       COUNT(*)
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 2;
```

Key Rule

```
WHERE → Filters Rows

HAVING → Filters Groups
```

3️⃣ HAVING Syntax

Basic syntax:

```
SELECT ColumnName,
       AggregateFunction(ColumnName)
FROM TableName
GROUP BY ColumnName
HAVING Condition;
```

Example

```
SELECT Department,
       AVG(Salary)
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

General Structure

```
SELECT
FROM
WHERE
```

```
GROUP BY
HAVING
ORDER BY
```

4️⃣ HAVING with COUNT()

Most common use case.

Example Table

Department
IT
IT
HR
HR
HR

Query:

```
SELECT Department,
       COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) >= 3;
```

Output:

Department	EmployeeCount
HR	3

Business Uses

- Departments with more than 10 employees
- Cities with more than 100 customers
- Products sold more than 1000 times

---

5 HAVING with SUM()

Filter groups based on totals.

Example

Department	Salary
IT	50000
IT	60000
HR	45000
HR	55000
HR	65000

Query:

```
SELECT Department,
       SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
HAVING SUM(Salary) > 150000;
```

Output:

Department	TotalSalary
HR	165000

Business Uses

- Cities with revenue above ₹1,00,000
- Products generating high sales
- Departments exceeding budget limits

---

6 HAVING with AVG()

Filter groups based on averages.

Query:

```sql
SELECT Department,
       AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 55000;
```

Output:

Department	AverageSalary
HR	55000

(Depending on dataset values)

Business Uses

- Departments with high average salaries
- Products with high average ratings
- Regions with high average revenue

7️⃣ HAVING with MIN() and MAX()

MAX Example

```sql
SELECT Department,
       MAX(Salary) AS HighestSalary
FROM Employees
GROUP BY Department
HAVING MAX(Salary) > 60000;
```

Output:

Only departments having salaries above 60000.

MIN Example

```sql
SELECT Department,
       MIN(Salary) AS LowestSalary
FROM Employees
```

```
GROUP BY Department
HAVING MIN(Salary) > 30000;
```

Output:

Departments where the lowest salary exceeds 30000.

---

8 HAVING with Multiple Conditions

You can combine conditions using:

```
AND
OR
NOT
```

Example

```
SELECT Department,
       COUNT(*) AS Employees,
       AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
HAVING COUNT(*) >= 2
AND AVG(Salary) > 50000;
```

Output:

Only groups satisfying both conditions.

Using OR

```
HAVING COUNT(*) > 10
OR AVG(Salary) > 50000
```

Using NOT

```
HAVING NOT COUNT(*) < 5
```

9️⃣ WHERE vs HAVING

This is one of the most important interview topics.

WHERE

Filters rows before grouping.

Example:

```
SELECT *
FROM Employees
WHERE Salary > 50000;
```

HAVING

Filters groups after grouping.

Example:

```
SELECT Department,
       AVG(Salary)
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

Comparison Table

Feature	WHERE	HAVING
Filters	Rows	Groups
Uses Aggregate Functions	❌	✅
Executes Before GROUP BY	✅	❌

Feature	WHERE	HAVING
Executes After GROUP BY	❌	✅

Example Using Both

```
SELECT Department,
       AVG(Salary)
FROM Employees
WHERE Salary > 30000
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

Explanation:

WHERE

Filters employees.

GROUP BY

Creates department groups.

HAVING

Filters department groups.

🔟 Query Execution Order

Query:

```
SELECT Department,
       AVG(Salary)
FROM Employees
WHERE Salary > 30000
GROUP BY Department
HAVING AVG(Salary) > 50000
ORDER BY AVG(Salary);
```

Actual Execution:

```
1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
```

Understanding this order is critical for advanced SQL.

1️⃣1️⃣ Common Errors

Using Aggregate in WHERE

Bad:

```
WHERE COUNT(*) > 2
```

Error.

Correct:

```
HAVING COUNT(*) > 2
```

Missing GROUP BY

Bad:

```
SELECT Department,
       COUNT(*)
FROM Employees
HAVING COUNT(*) > 2;
```

Usually incorrect logic.

Wrong Aggregate Usage

Bad:

```
HAVING EmployeeName = 'Kunal'
```

Use WHERE instead.

Using HAVING Instead of WHERE

Bad:

```
HAVING Salary > 50000
```

Better:

```
WHERE Salary > 50000
```

1️⃣2️⃣ Best Practices

Use WHERE First

Filter rows early.

Good:

```
WHERE Salary > 30000
```

Use HAVING Only for Aggregates

Good:

```
HAVING AVG(Salary) > 50000
```

Use Meaningful Aliases

Good:

```
AVG(Salary) AS AverageSalary
```

Keep Conditions Readable

Use proper formatting.

Combine WHERE and HAVING

Improves performance.

Professional Example

```
SELECT Department,
       COUNT(*) AS EmployeeCount,
       AVG(Salary) AS AverageSalary
FROM Employees
WHERE Salary > 30000
GROUP BY Department
HAVING COUNT(*) >= 2
ORDER BY AverageSalary DESC;
```

1️⃣3️⃣ Business Analytics Examples

Departments with More Than 5 Employees

```
SELECT Department,
       COUNT(*)
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 5;
```

Cities Generating Revenue Above ₹1 Lakh

```
SELECT City,
       SUM(Revenue)
FROM Sales
GROUP BY City
HAVING SUM(Revenue) > 100000;
```

Products with Average Rating Above 4

```
SELECT ProductName,
       AVG(Rating)
FROM Reviews
GROUP BY ProductName
HAVING AVG(Rating) > 4;
```

Customers with Multiple Orders

```
SELECT CustomerID,
       COUNT(*)
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) > 1;
```

KPI Dashboard Query

```
SELECT Department,
```

```
        COUNT(*) AS Employees,
        AVG(Salary) AS AvgSalary,
        SUM(Salary) AS TotalSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

📝 Summary

In this module, you learned:

✅ HAVING Clause

✅ HAVING with COUNT()

✅ HAVING with SUM()

✅ HAVING with AVG()

✅ HAVING with MIN()

✅ HAVING with MAX()

✅ Multiple Conditions

✅ WHERE vs HAVING

✅ Query Execution Order

✅ Business Analytics Queries

🧠 Practice Questions

Theory

1. What is HAVING?
2. Why is HAVING needed?
3. Difference between WHERE and HAVING?
4. Can HAVING use aggregate functions?
5. Can WHERE use aggregate functions?
6. When does HAVING execute?
7. What is query execution order?
8. How does HAVING work with COUNT()?

9. How does HAVING work with SUM()?
10. Why combine WHERE and HAVING?

Practical Exercises

Task 1

Show departments having:

```
COUNT(*) > 2
```

Task 2

Show departments where:

```
AVG(Salary) > 50000
```

Task 3

Show cities with:

```
SUM(SalesAmount) > 100000
```

Task 4

Show customers with:

```
COUNT(*) > 1
```

Task 5

Use both:

```
WHERE
HAVING
```

in a single query.

Task 6

Find departments where:

```
MIN(Salary) > 30000
```

Challenge Project

Build an Employee Analytics Report showing:

- Departments with more than 3 employees
- Average salary above ₹50,000
- Total salary expense above ₹1,00,000

Use:

- GROUP BY
- HAVING
- COUNT()
- AVG()
- SUM()

in a single query.

🚀 Next Module

➞ Module 13: UPDATE Statement

Topics Covered:

- UPDATE Syntax
- Updating Single Records
- Updating Multiple Records
- UPDATE with WHERE
- UPDATE with Expressions
- UPDATE Using JOIN
- Common Mistakes
- Best Practices